

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: CSA04

COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments. (BL2, BL3)*

QUESTIONS: 5

Total Marks:100

Annexure A

Concept Mapping

Code of Conduct:

I M. Pranay (192525382)_certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

M. Pranay
Signature of the student:

Annexure A

Concept Mapping

Question 1

Show a multi-level concept map integrating:

Operating System Types → OS Structures → System Calls → OS Functions.

Your map must show cross-links (e.g., how OS structure influences system call implementation). **(BL2)**

Question 2

Construct a comprehensive concept map representing:

Process Management, Memory Management, File System, and I/O System.

Include interdependencies and indicate where system calls interact with each component. **(BL3)**

Question 3

Sketch a comparative concept map for:

Batch OS, Time-Sharing OS, Distributed OS, and Real-Time OS.

Highlight differences, similarities, advantages, and limitations using linking phrases. **(BL2)**

Question 4

Illustrate a dynamic concept map showing the execution flow:

User Program → API → System Call Interface → Kernel → Hardware → Response back to User.

Include feedback loops and control flow paths. **(BL3)**

Question 5

Sketch a hierarchical + functional concept map for system calls:

Process, File, Device, and Information system calls.

Also show how they relate to OS services like scheduling, memory allocation, and device handling. **(BL3)**

RUBRICS FOR CALCULATION

Concept Mapping					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

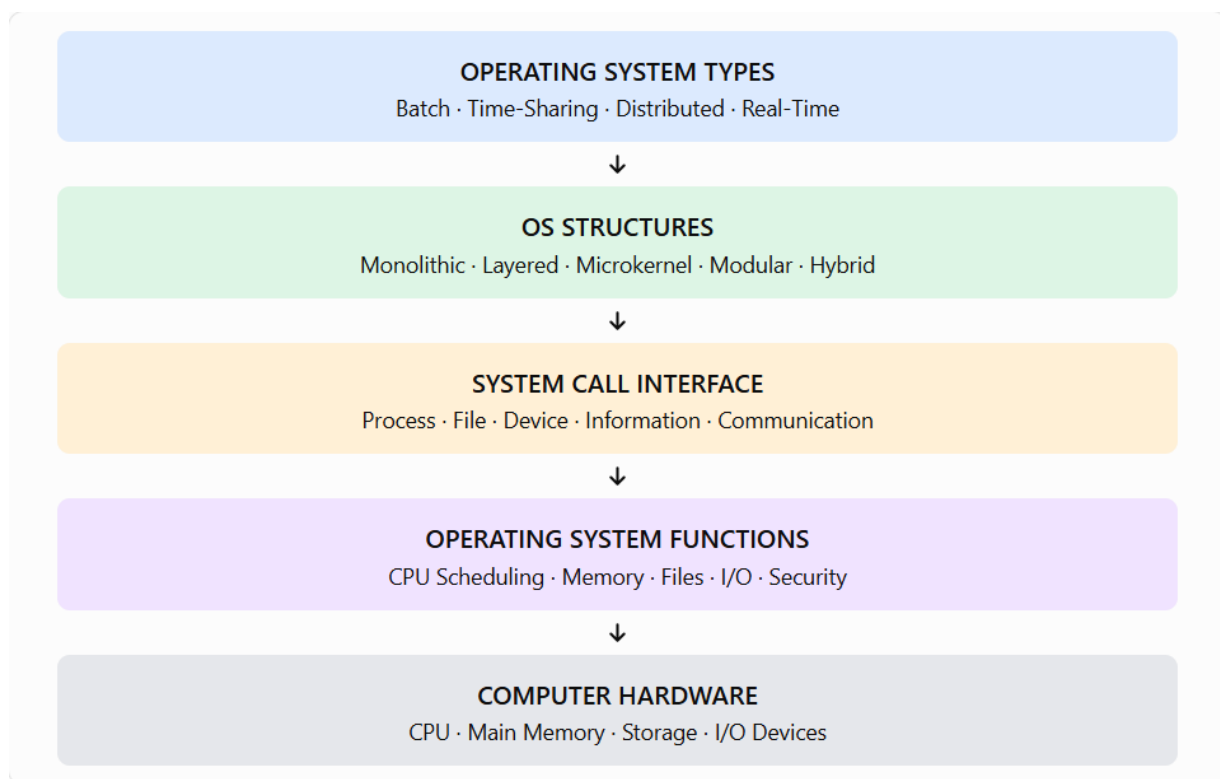
ANSWERS

1. An Operating System (OS) is system software that acts as an interface between the user and computer hardware. It manages resources and provides services to application programs.

The four main concepts are connected as follows:

- OS Types: Batch OS, Time-Sharing OS, Distributed OS, Real-Time OS.
- OS Structures: Monolithic, Layered, Microkernel, Modular, Hybrid.
- System Calls: Provide an interface through which programs request services from the OS.
- OS Functions: Process management, memory management, file management, device management, security, and resource allocation.

Block diagram



Cross-link: OS structure influences how system calls are implemented. For example, a monolithic kernel handles many services inside the kernel, while a microkernel moves some services into user-space processes and uses message passing for communication.

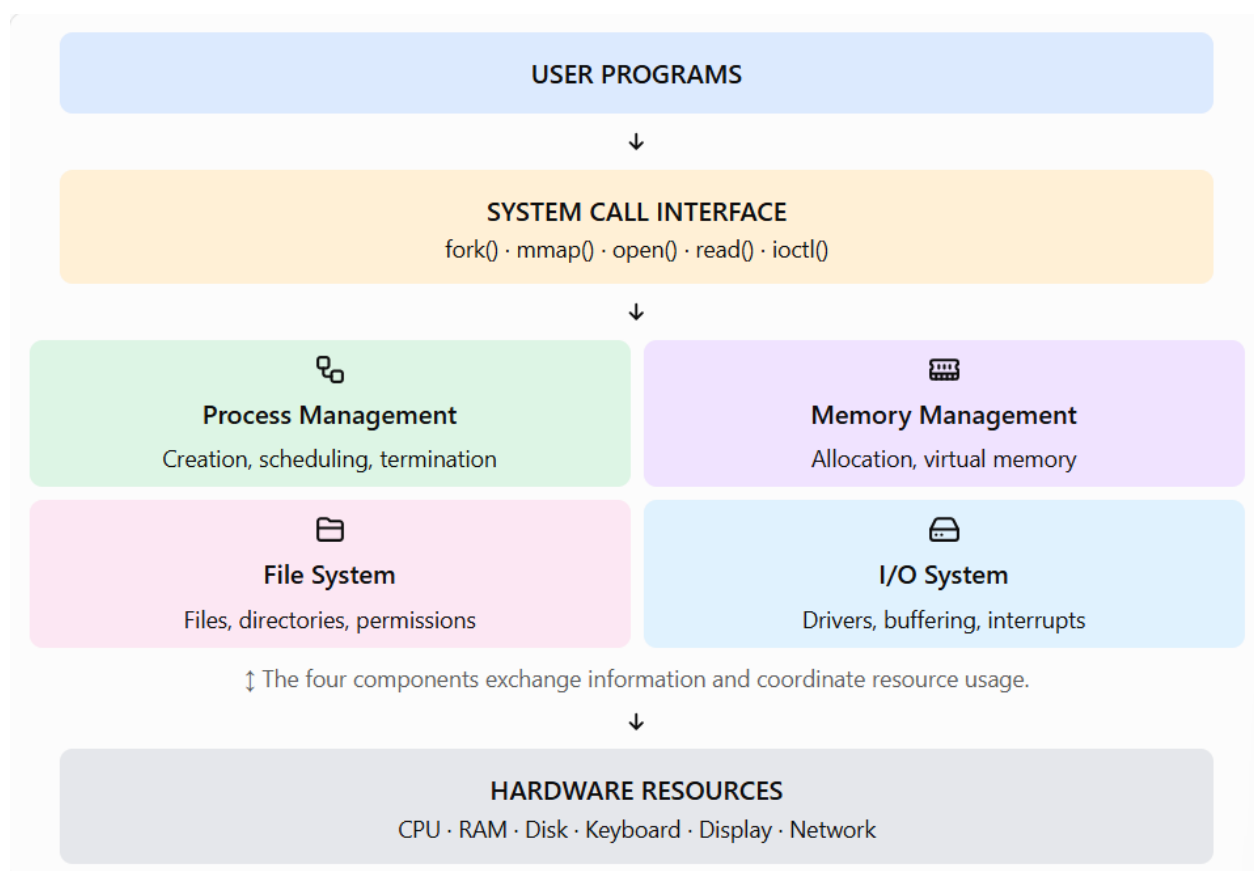
Conclusion: OS types describe how operating systems are used, OS structures describe their internal organization, system calls provide access to OS services, and OS functions manage hardware and software resources.

2.The four major operating system components work together to execute programs and manage resources.

1. **Process Management:** Creates and terminates processes, schedules the CPU, and handles synchronization and inter-process communication.
2. **Memory Management:** Allocates and deallocates memory, tracks memory usage, and supports virtual memory.
3. **File System:** Creates, reads, writes, deletes, and organizes files and directories.
4. **I/O System:** Controls input/output devices through device drivers, buffering, caching, and interrupt handling.

System calls connect user programs to these components. Examples include `fork()` and `exec()` for process operations on Unix-like systems, `mmap()` for memory mapping, `open()` and `read()` for files, and `ioctl()` for device control.

Block diagram



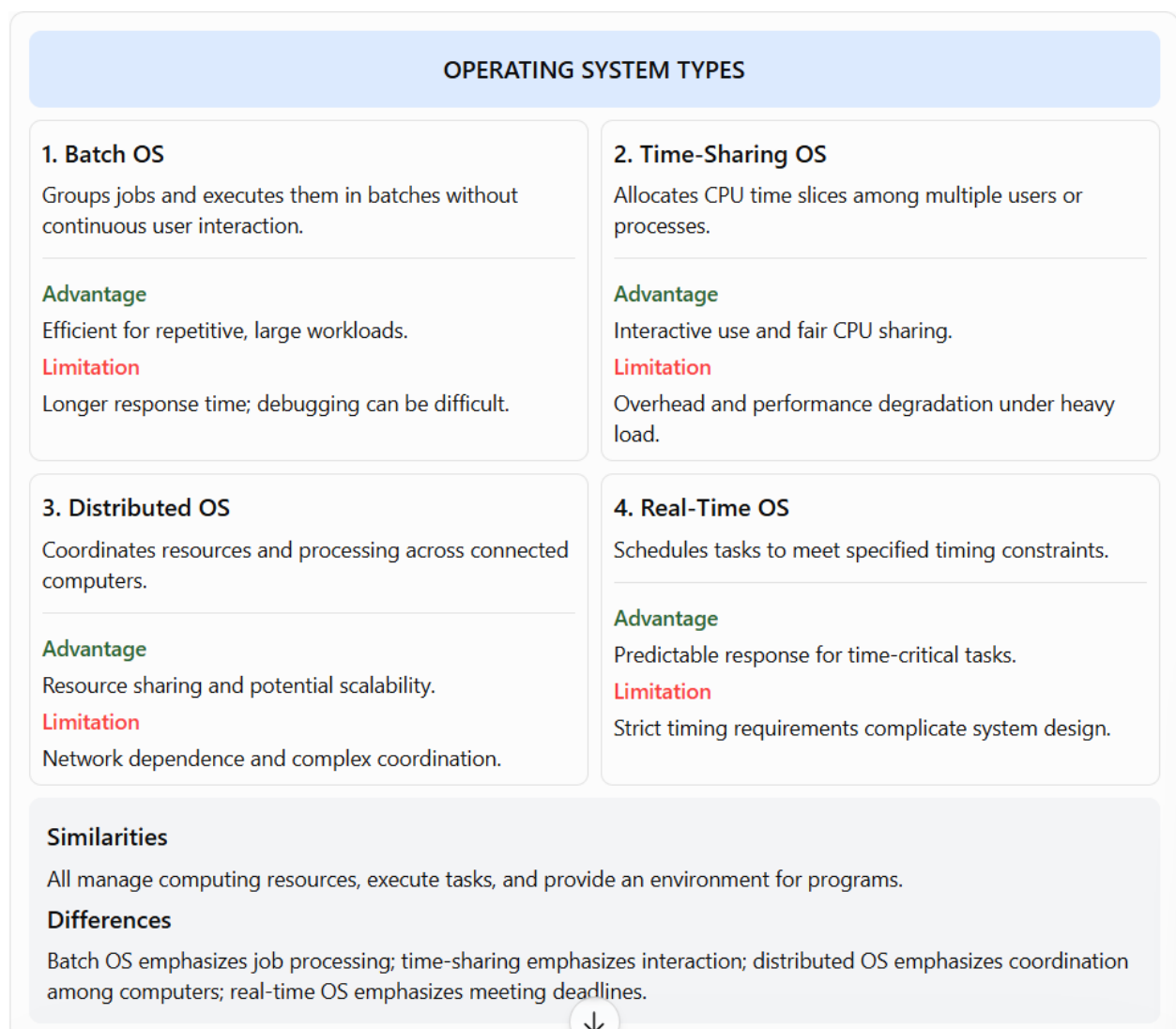
Interdependencies

- Process management requests memory allocation before a process executes.
- Memory management uses files and storage to support mechanisms such as memory-mapped files and swapping.
- The file system relies on I/O operations to read from and write to storage devices.

- The I/O system uses interrupts to notify the CPU when device operations complete.
- System calls provide controlled access to all four components.

Conclusion: These components are interdependent. Their coordination allows programs to execute efficiently while sharing CPU time, memory, files, and I/O devices.

3. Operating systems can be classified according to how they execute jobs, share resources, connect computers, and respond to deadlines.



Comparison table

Feature	Batch OS	Time-Sharing OS	Distributed OS	Real-Time OS
Main objective	Batch execution	Interactive access	Distributed resource use	Deadline satisfaction
Processing	Groups of jobs	CPU time slices	Multiple connected systems	Priority/deadline-aware scheduling
User interaction	Low during execution	High	Depends on system	Depends on application
Typical application	Payroll processing	Multi-user computing	Distributed computing	Embedded control systems

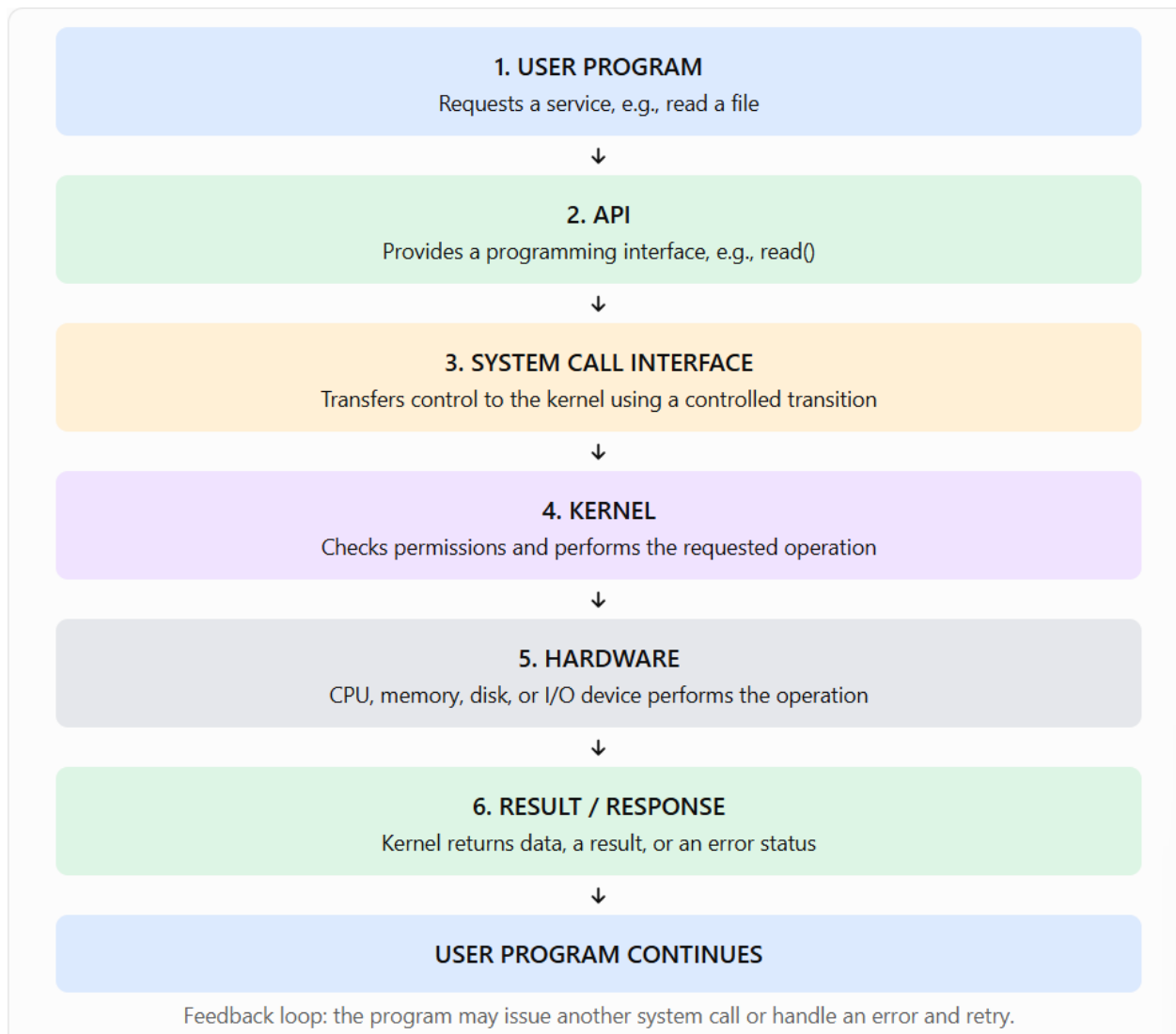
Linking phrases

- Batch OS groups and executes jobs.
- Time-sharing OS shares CPU time among users and processes.
- Distributed OS coordinates resources across computers.
- Real-time OS schedules tasks according to timing constraints.
- All four manage resources and execute programs, but they differ in their primary objectives.

Conclusion: Each OS type serves different computing requirements. The appropriate type depends on workload, interactivity, distribution needs, and timing constraints.

4. A system call allows a user program to request a service from the operating system kernel. The request passes through an API and system call interface before the kernel performs the required operation. The result or error status is then returned to the user program.

Block diagram



Execution steps

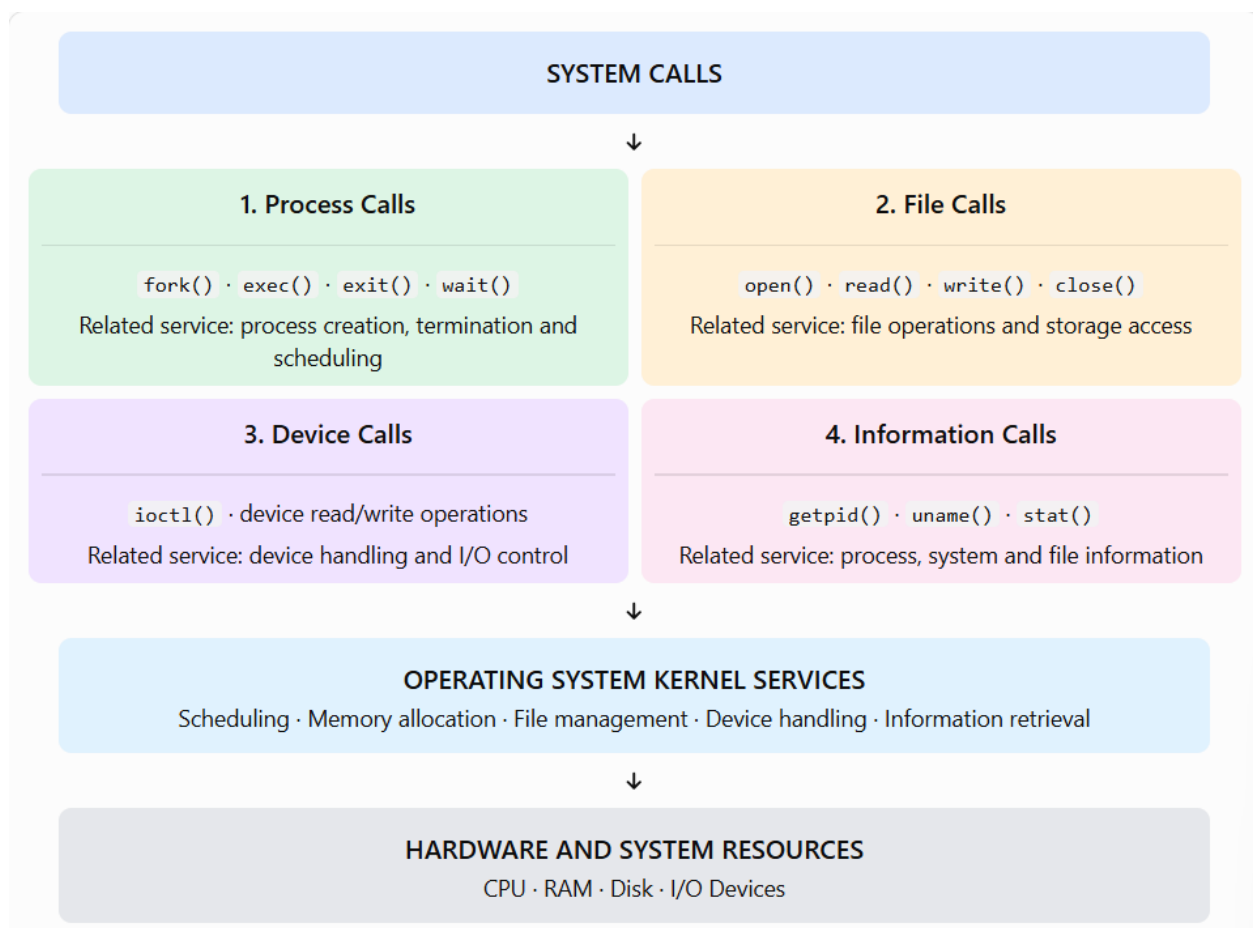
1. The user program requests an operation, such as reading a file.
2. The API provides a convenient interface to the required service.
3. The system call interface transfers execution from user mode to kernel mode.
4. The kernel validates the request and checks access permissions.
5. The kernel communicates with hardware or performs the operation using existing resources.
6. The result or error status is returned through the system call interface.
7. The program processes the response and may continue, retry, or issue another request.

Feedback and control flow: If an operation fails, the program may handle the error and make another request. If a device operation completes later, an interrupt can notify the operating system, which then completes or resumes the appropriate work.

Conclusion: The system call mechanism provides controlled access to kernel services while protecting hardware and system resources from direct, unrestricted user-program access.

5. System calls are commonly classified into four categories for this question: process, file, device, and information system calls. Each category provides services that help the operating system manage processes and hardware resources.

Block diagram



Functions of each system call category

Category	Main purpose	Example
Process	Create, execute, wait for, and terminate processes	fork(), exec(), exit()
File	Create, access, modify, and close files	open(), read(), write()
Device	Request device operations and control device-specific functions	ioctl()
Information	Obtain process, file, or system information	getpid(), uname(), stat()

Note: System call names and availability vary across operating systems. These examples primarily use Unix/Linux conventions.

Relationship with OS services

- Scheduling: Process calls enable process creation and termination; the scheduler determines which runnable process receives CPU time.
- Memory allocation: Process execution and memory-related operations require the kernel to manage address spaces and memory resources.
- Device handling: Device calls allow programs to request controlled I/O operations through the kernel and device drivers.
- Information retrieval: Information calls return system, process, or file metadata to programs.
- File management: File calls connect programs to file-system services and storage operations.

Conclusion: System calls form the interface between application programs and operating system services. Their classification helps organize how the kernel manages processes, files, devices, and system information.